PLATEFORME DE SUPERVISION INFOGÉRANCE

Documentation Technique — Procédure d'Installation & d'Exploitation

Réseau • Systèmes • Développement PHP / Python • Automatisation

Document rédigé dans le cadre d'un projet de portfolio.
Toutes les données sensibles (adresses IP, identifiants, communautés SNMP, mots de passe) ont été anonymisées.

Sommaire

1. Introduction générale

2. Architecture globale

3. Partie 1 — Scripts de collecte (Python)

3.1 Emplacement et rôle des fichiers

3.2 Scanner Switch — SNMP (Alcatel-Lucent)

3.3 Scanner Serveur — Windows (WinRM)

3.4 Scanner Firewall — FortiGate (SNMP)

4. Partie 2 — Modules PHP

4.1 Supervision Web — cron_web.php

4.2 Module de messagerie (Mails)

5. Partie 3 — Automatisation & Exploitation

5.1 Services Systemd

5.2 Planification Cron

5.3 Commandes courantes d'exploitation

5.4 Dépannage rapide

6. Modèle de données (synthèse)

7. Sécurité & contrôle d'accès

8. Synthèse

1. Introduction générale

Ce document présente l'architecture, l'installation et l'exploitation d'une plateforme de supervision infogérance développée pour centraliser la surveillance d'une infrastructure réseau et applicative (serveurs, switchs, pare-feu, sites web).

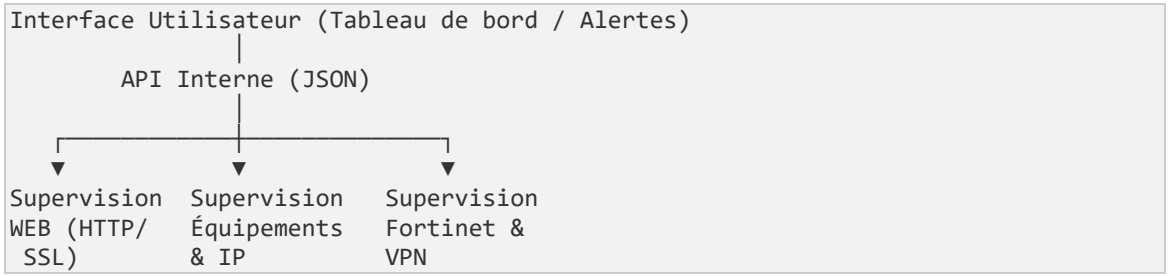
La solution repose sur trois grands blocs techniques, traités dans cet ordre logique tout au long du document :

- Les scripts de collecte Python (scanners réseau : switch, serveur, firewall) — le cœur de la remontée de données ;
- Les modules PHP (interface web, tests de disponibilité, module de messagerie) — la couche applicative et de restitution ;
- L'automatisation et l'exploitation quotidienne (Systemd, Cron, procédures de dépannage) — le maintien en conditions opérationnelles.

> *⚠ Par souci de confidentialité, toutes les adresses IP, communautés SNMP, identifiants et mots de passe présents dans les captures d'origine ont été remplacés par des valeurs génériques ou masqués.*

2. Architecture globale

La plateforme s'articule autour d'une base de données centrale (MySQL/MariaDB) alimentée par des scripts de collecte autonomes, et restituée via une interface web PHP.



Chaque équipement dispose de son propre script de collecte, exécuté en continu en arrière-plan via un service système dédié.

3. Partie 1 — Scripts de collecte (Python)

Cette partie regroupe l'ensemble des scripts responsables de la remontée des métriques : switchs réseau, serveurs Windows et pare-feu Fortinet.

3.1 Emplacement et rôle des fichiers

Composant	Emplacement	Rôle
Racine application Python	/opt/supervision/	Dossier racine des scripts de collecte
Scanner Switch	/opt/supervision/collector/switch/scanner.py	Collecte des données réseau (switchs)
Scanner Serveur	/opt/supervision/collector/server/scanner.py	Collecte des métriques serveurs (WinRM)
Scanner Firewall	/opt/supervision/collector/firewall/scanner.py	Collecte de l'état des pare-feux (SNMP)
Interface Web	/var/www/html/	Site web d'affichage de la supervision
Services Systemd	/etc/systemd/system/*.service	Configuration du démarrage automatique

3.2 Scanner Switch — SNMP (Alcatel-Lucent)

Le scanner switch interroge les équipements réseau en SNMP v2c (MIB-II standard + MIB propriétaire du constructeur) pour récupérer l'état de santé et le statut des ports.

Paramètres réseau & timeouts (en tête de script)

Variable	Valeur par défaut	Description
SNMP_COMMUNITY	*[COMMUNAUTÉ MASQUÉE]*	Communauté SNMP en lecture. À modifier si elle change sur les équipements.
SNMP_PORT	161	Port UDP d'écoute SNMP.
SNMP_TIMEOUT	3 s	Délai d'attente par tentative (à augmenter sur liaisons WAN/VPN).
SNMP_RETRIES	2	Nombre de re-essais avant abandon.

Référentiel des OID SNMP

Les OID normalisés (MIB-II) sont communs à tous les constructeurs : nom d'hôte, description système, uptime, nom et statut des interfaces. Les OID de santé (CPU, RAM, température) dépendent en revanche de la branche propriétaire du constructeur (ici Alcatel-Lucent). Ces identifiants sont documentés publiquement par le fabricant et communs à tout équipement de même gamme ; ils ne présentent pas de caractère confidentiel.

OID	Portée	Donnée
1.3.6.1.2.1.1.5.0 / 1.3.6.1.2.1.1.1.0 / 1.3.6.1.2.1.1.3.0	Standard MIB-II	sysName / sysDescr / sysUpTime — nom, modèle/OS, uptime
1.3.6.1.2.1.31.1.1.1.1 / 1.3.6.1.2.1.2.2.1.2 / 1.3.6.1.2.1.2.2.1.8	Standard MIB-II	ifName / ifDescr / ifOperStatus — nom et état opérationnel des ports
1.3.6.1.4.1.6486.800.1.2.1.16.1.1.1.14.0	Fabricant (Alcatel-Lucent)	OID_ALCATEL_CPU — % d'utilisation processeur
1.3.6.1.4.1.6486.800.1.2.1.16.1.1.1.9.0	Fabricant (Alcatel-Lucent)	OID_ALCATEL_RAM — % d'utilisation mémoire
1.3.6.1.4.1.6486.800.1.2.1.16.1.1.1.3.0	Fabricant (Alcatel-Lucent)	OID_ALCATEL_TEMP — température du châssis (°C)

Gestion des exceptions de modèles

Certains modèles d'entrée de gamme n'exposent pas les OID de santé propriétaires ; les interroger provoque des timeouts inutiles. Une liste blanche d'exclusion permet d'écarter ces modèles :

```
MODELES_SANS_SANTE = ["<référence_modèle>"]
```

Pour ajouter un nouveau modèle non compatible, il suffit d'ajouter sa référence dans ce tableau. La détection de marque/modèle se fait par analyse de la chaîne sysDescr ; un modèle non reconnu peut être ajouté via un bloc conditionnel dédié.

Statuts de ports

Valeur	Statut	Signification
1	UP	Port connecté et actif
2	DOWN	Port déconnecté
3	TEST	Mode test
4	UNKNOWN	Statut inconnu
5	DORMANT	En attente d'événement
6	NOT PRESENT	Module ou SFP retiré
7	LOWER LAYER DOWN	Lien physique bas

Un mécanisme de nettoyage par expression régulière normalise les noms de ports lorsque le constructeur y concatène un statut (ex. « Port 1/1/1 - UP » → « Port 1/1/1 »).

3.3 Scanner Serveur — Windows (WinRM)

Le scanner serveur collecte les métriques système des machines Windows via le protocole WinRM, en trois étapes :

- Pré-vérification réseau : test d'ouverture du port TCP 5985 avant toute authentification, pour éviter les timeouts longs sur une machine éteinte ;
- Exécution distante (WinRM / NTLM) : requêtes PowerShell légères via l'interface CIM (Get-CimInstance) ;

- Formatage JSON : le résultat (CPU, RAM, disque) est structuré en JSON puis converti en dictionnaire exploitable côté Python.

Variables de configuration

Variable	Valeur	Description
CONFIG_FILE	/opt/supervision/config/windows.conf	Chemin du fichier d'identifiants Windows
PORT_WINRM	5985	Port HTTP WinRM standard (5986 en HTTPS)
TIMEOUT_PORT	2 s	Délai du test de socket initial
TIMEOUT_OPERATION	10 s	Temps max pour l'exécution de la commande distante
TIMEOUT_READ	15 s	Temps max de lecture de la réponse (doit rester > TIMEOUT_OPERATION)

Fichier de configuration windows.conf

Les identifiants d'accès Windows sont externalisés dans un fichier INI dédié, séparé du code source :

```
[windows]
utilisateur = [DOMAINE MASQUÉ]\[UTILISATEUR MASQUÉ]
mot_de_passe = [MOT DE PASSE MASQUÉ]
```

> *⚠ Le fichier windows.conf contient des secrets : il doit être exclu de tout dépôt Git et protégé par des droits de lecture restreints (chmod 600).*

3.4 Scanner Firewall — FortiGate (SNMP)

La supervision du pare-feu repose sur une collecte SNMP asynchrone en boucle continue, interrogeant la MIB propriétaire Fortinet ainsi que la MIB-II standard.

```
[FortiGate] --(SNMP UDP 161)--> snmp_client.py
                                        |
                                        ▼
                        scanner.py (boucle & seuils) <--> Base de données
```

Paramètres réseau SNMP

Variable	Valeur	Description
SNMP_COMMUNITY	*[COMMUNAUTÉ MASQUÉE]*	Communauté SNMP v2c du FortiGate
SNMP_PORT	161	Port UDP SNMP
SNMP_TIMEOUT	3 s	Délai d'attente par requête
SNMP_RETRIES	2	Tentatives de reconnexion

Paramètres de supervision

Variable	Valeur	Description
IP_FIREWALL	*[IP MASQUÉE]*	Adresse IP du pare-feu supervisé
NOM_FIREWALL	[Modèle masqué]	Libellé par défaut si la résolution DNS échoue

Variable	Valeur	Description
INTERVALLE_METRIQUES	2 s	Pause entre chaque cycle de contrôle
TYPE_FIREWALL / SITE_ID	—	Identifiants de rattachement (table & site)

Seuils d'alerte

Statut	Niveau	Condition	Action
1	OK	CPU < 70 % ET RAM < 70 % ET joignable	Suppression de l'alerte active
2	WARNING	CPU ou RAM entre 70 % et 84 %	Création / mise à jour de l'alerte
3	CRITICAL	Injoignable OU CPU/RAM ≥ 85 %	Alerte + journalisation incident

Correspondance des OID SNMP (Fortinet)

Comme pour les switchs, ces OID sont ceux de la MIB propriétaire Fortinet, documentés publiquement et communs à tout FortiGate — ils ne présentent pas de caractère confidentiel.

OID	Donnée
1.3.6.1.2.1.1.5.0	Nom d'hôte (sysName)
1.3.6.1.4.1.12356.101.4.1.1.0	Version firmware
1.3.6.1.2.1.47.1.1.1.1.11.1	Numéro de série
1.3.6.1.4.1.12356.101.4.1.20.0	Uptime système
1.3.6.1.4.1.12356.101.4.1.3.0	Charge CPU (%)
1.3.6.1.4.1.12356.101.4.1.4.0	Usage RAM (%)
1.3.6.1.4.1.8.0	Sessions actives
1.3.6.1.4.1.12356.101.4.2.1.0	Température (°C)
1.3.6.1.4.1.12356.101.12.2.2.1.20	Statut tunnels VPN IPsec
1.3.6.1.4.1.12356.101.12.2.2.1.2	Noms phase 1 IPsec
1.3.6.1.4.1.12356.101.12.2.3.1.0	Nombre total d'utilisateurs VPN SSL
1.3.6.1.4.1.12356.101.12.2.3.1.2	Utilisateurs connectés SSL
1.3.6.1.4.1.12356.101.12.2.3.1.3	IP sources SSL
1.3.6.1.4.1.12356.101.12.2.3.1.4	IP tunnels SSL

Chaque cycle de scrutation met à jour les tables equipements, firewall, firewall_vpn_ssl_sessions, firewall_vpn_ipsec_phase2 et alertes.

4. Partie 2 — Modules PHP

Cette partie couvre la couche applicative : la vérification automatique des sites web supervisés et le module d'envoi de courriels.

4.1 Supervision Web — cron_web.php

Pour chaque site enregistré, le script exécute trois contrôles indépendants, chacun valant un point sur trois :

- Accessibilité HTTP/HTTPS : code de retour compris entre 200 et 399 (via cURL, suivi des redirections activé) ;
- Certificat SSL : ouverture d'un flux TLS sur le port 443 et vérification que la date d'expiration du certificat est postérieure à l'heure actuelle ;
- Intégrité du contenu : recherche d'un mot-clé métier attendu dans le code HTML renvoyé.

Logique d'alerte

- Chaque site est identifié dans les alertes par un tag [WEB:ID_SITE] ;
- Les alertes web ont equipement_id = NULL et statut_id = 3 (critique) ;
- Si une alerte active existe déjà pour ce tag, elle est mise à jour plutôt que dupliquée ;
- Dès que le score repasse à 3/3, l'alerte est automatiquement levée (suppression).

Extrait de code (fonctions de test)

```
function testerSiteEtContenu($url, $mot_cle) {
    $ch = curl_init($url);
    curl_setopt_array($ch, [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_FOLLOWLOCATION => true,
        CURLOPT_TIMEOUT => 5,
    ]);
    $contenu   = curl_exec($ch);
    $http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
    curl_close($ch);

    $en_ligne   = ($http_code >= 200 && $http_code < 400);
    $contenu_ok = $en_ligne && !empty($mot_cle)
        && mb_strpos($contenu, $mot_cle) !== false;

    return compact('en_ligne', 'http_code', 'contenu_ok');
}

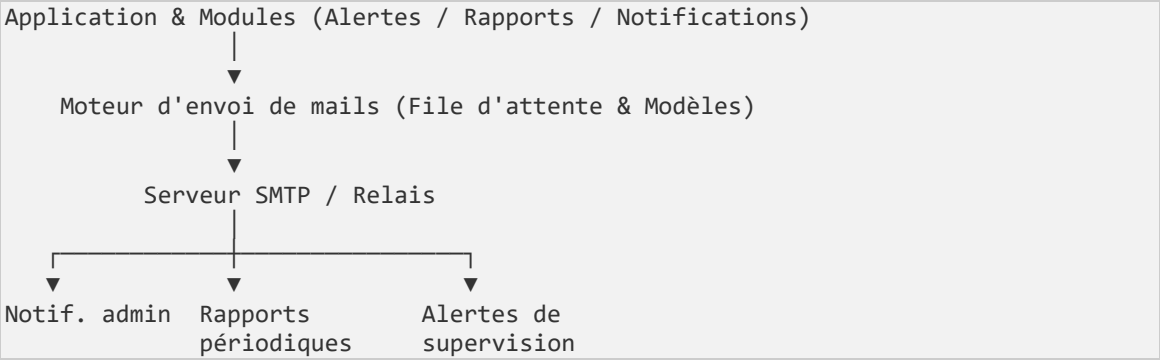
function testerSSL($url) {
    $host   = parse_url($url, PHP_URL_HOST) ?: $url;
    $client = @stream_socket_client("ssl://$host:443", $errno, $errstr, 5,
                STREAM_CLIENT_CONNECT,
                stream_context_create(["ssl" => ["capture_peer_cert" => true]]));
    if (!$client) return 0;

    $cert =
stream_context_get_params($client)["options"]["ssl"]["peer_certificate"] ?? null;
    $info = $cert ? openssl_x509_parse($cert) : null;
    return (time() < ($info['validTo_time_t'] ?? 0)) ? 1 : 0;
}
```

4.2 Module de messagerie (Mails)

Le module de messagerie centralise l'envoi de tous les courriels sortants de la plateforme : notifications d'alertes, rapports périodiques, réinitialisations de mot de passe.

Architecture d'envoi



Fonctionnement de la file d'attente

- Envoi asynchrone : les courriels sont mis en file pour ne jamais bloquer l'interface utilisateur ;
- Priorité haute : alertes de sécurité / incidents → envoi immédiat ;
- Priorité normale : rapports et récapitulatifs → envoi différé ;
- Mécanisme de reprise (retry) : plusieurs tentatives espacées en cas d'échec SMTP temporaire avant de déclarer le message en erreur.

Sécurité des envois

- Chiffrement des communications sortantes via TLS/SSL ;
- Authentification SMTP par identifiants dédiés (non stockés en clair dans le code applicatif) ;
- Alignement SPF/DKIM pour limiter le risque de classement en courrier indésirable.

Matrice des flux réseau requis

Port	Protocole	Usage
587	TCP	Envoi SMTP sécurisé avec chiffrement opportuniste (STARTTLS)
465	TCP	Envoi SMTP sécurisé via tunnel SSL/TLS direct
25	TCP	Relais SMTP interne (réseau local uniquement, sans chiffrement)

Procédure en cas de non-réception d'un mail

- 1. Vérifier le statut du message dans l'interface (En attente / Envoyé / Échec / Rejeté) ;
- 2. Consulter le journal dédié mail_error.log pour identifier la cause du rejet ;
- 3. Relancer l'envoi si le problème est temporaire, ou corriger l'adresse destinataire si le problème vient du destinataire.

5. Partie 3 — Automatisation & Exploitation

Cette dernière partie décrit comment les scripts sont exécutés en continu (Systemd), planifiés (Cron), supervisés et dépannés au quotidien.

5.1 Services Systemd

Chaque scanner Python (switch, server, firewall) est packagé comme un service Systemd indépendant, démarré automatiquement au boot du serveur.

Modifier le code d'un scanner

- Éditer le fichier concerné, par exemple : /opt/supervision/collector/switch/scanner.py ;
- Enregistrer les modifications ;
- Redémarrer uniquement le service impacté :

```
sudo systemctl restart switch   # ou server / firewall
```

Modifier la configuration Systemd (chemin, commande de lancement)

Toute modification d'un fichier .service situé sous /etc/systemd/system/ nécessite un rechargement de la configuration avant redémarrage :

```
sudo systemctl daemon-reload
sudo systemctl restart switch server firewall
```

> *⚠ Oublier le daemon-reload après modification d'un fichier .service est une source fréquente de service qui redémarre avec l'ancienne configuration.*

Résilience

Les services sont configurés avec Restart=always et RestartSec=5 : en cas d'erreur ponctuelle (équipement injoignable, exception Python), le service relance automatiquement une tentative 5 secondes plus tard, sans intervention manuelle.

5.2 Planification Cron

La supervision web (cron_web.php) est automatisée via une tâche planifiée exécutée toutes les 5 minutes :

```
*/5 * * * * /usr/bin/php /chemin/vers/le/projet/cron_web.php >/dev/null 2>&1
```

Pour désactiver temporairement cette vérification automatique, il suffit de retirer ou commenter cette ligne via crontab -e.

Création d'un compte utilisateur

La création des comptes se fait via le script create_user.php exécuté manuellement (ouverture dans un éditeur, saisie du nom d'utilisateur, du mot de passe et du rôle). Ce fichier est l'unique point d'entrée assurant le hachage sécurisé du mot de passe avant insertion en base ; il ne doit pas être supprimé.

5.3 Commandes courantes d'exploitation

Vérifier l'état des services

```
sudo systemctl status switch server firewall
```

Consulter les logs en temps réel

```
sudo journalctl -u switch -f
sudo journalctl -u server -f
sudo journalctl -u firewall -f
```

Redémarrer / arrêter

```
sudo systemctl restart switch server firewall
sudo systemctl stop switch
```

5.4 Dépannage rapide

Symptôme	Vérification	Action
Service en état failed	journalctl -u <service> -n 50	Identifier l'erreur Python (syntaxe, équipement injoignable) et corriger
Comportement après crash	Restart=always / RestartSec=5	Le service retente automatiquement au bout de 5 s
Erreurs de droits	Utilisateur d'exécution des services	S'assurer que l'utilisateur applicatif conserve les droits de lecture sur /opt/supervision

6. Modèle de données (synthèse)

La base de données centralise l'ensemble des équipements, de leurs métriques et des alertes associées.

Table / fichier	Contenu principal
users	Comptes utilisateurs, mots de passe hachés, rôles
sites	Référentiel des sites physiques
equipements	Base commune des équipements (nom, IP, marque, dernière vérification)
serveurs / switchs / firewall	Métriques spécifiques par type d'équipement
ports	Ports réseau associés aux switchs
firewall_vpn_ssl_sessions / firewall_vpn_ipsec_phase2	Sessions et tunnels VPN
supervision_web	Sites web suivis, URLs, mots-clés, statuts HTTP/SSL
alertes	Journal des alertes en cours et passées
incident.log	Historique texte indépendant des incidents

7. Sécurité & contrôle d'accès

- Authentification par identifiant et mot de passe haché (algorithme fort) ;
- Rôle « utilisateur standard » : consultation seule du tableau de bord ;
- Rôle « administrateur » : gestion complète (alertes, pare-feu, supervision web, recherche) ;
- Isolation des accès aux fichiers par liste blanche stricte de routage.

Matrice des flux réseau — plateforme de supervision

Port	Protocole	Usage
80 / 443	TCP	Contrôle de disponibilité web et validation des certificats SSL
5985	TCP	Test WinRM (vérification directe des serveurs Windows)
3306	TCP	Connexion à la base de données MySQL / MariaDB
161	UDP	Collecte SNMP (switchs et pare-feu)

8. Synthèse

Cette documentation retrace l'ensemble de la chaîne technique d'un projet de supervision infogérance : de la collecte bas niveau (SNMP, WinRM) jusqu'à la restitution applicative (PHP, alertes, mails), en passant par l'automatisation de l'exploitation (Systemd, Cron). Elle illustre une approche complète mêlant réseau, système, développement et exploitation.